

Datové struktury a algoritmy

Přednáška 6, poznámky

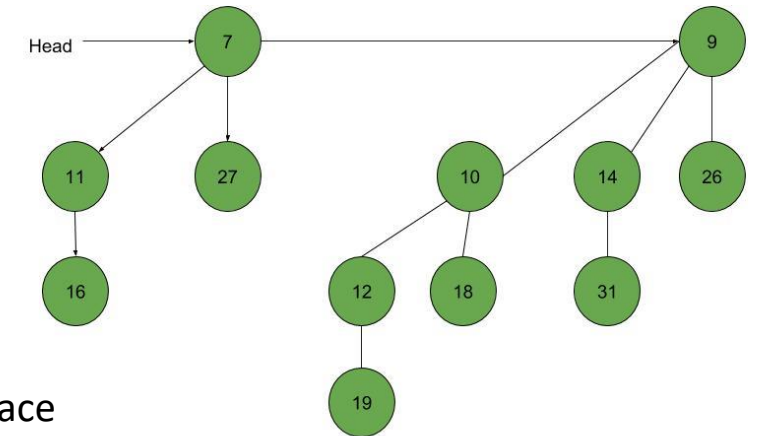
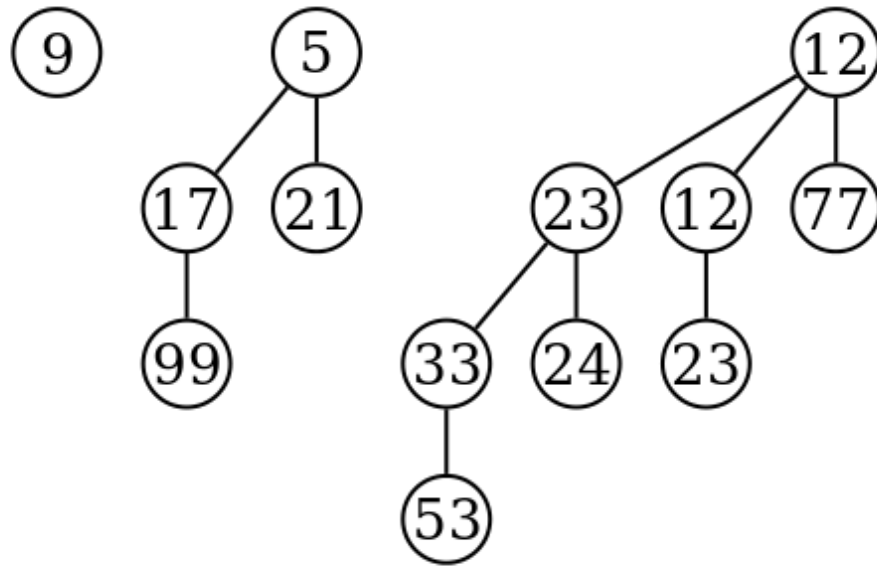
Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

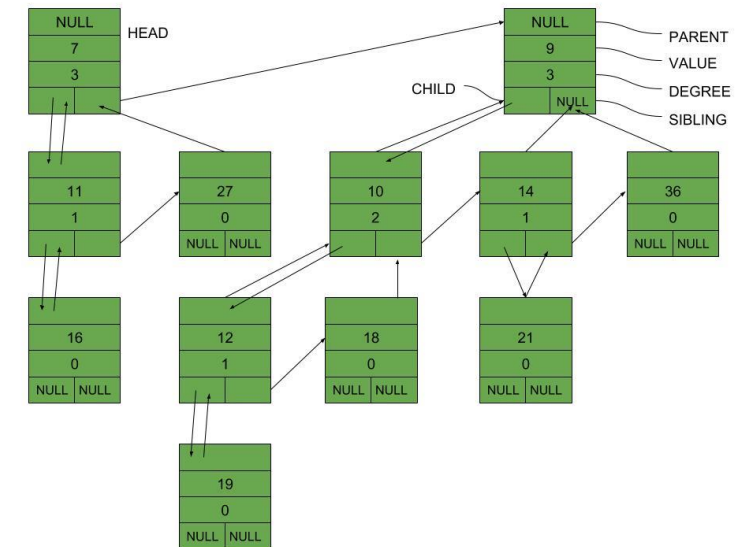
Témata

- Prezentace (10 studentů)
 - Select sort, Insert sort, Bubble sort, Shell sort, Heap sort, Quick sort, Merge sort
- Bonifikovaný kvíz 1: Stromy a jejich obecné vlastnosti
- Halda a prioritní fronta (dokončení)
- Bonifikovaný kvíz 2 : Halda a prioritní fronta
- Algoritmy řazení
 - Všeobecný úvod
 - Algoritmy vnitřního řazení

Binomiální halda



Reprezentace



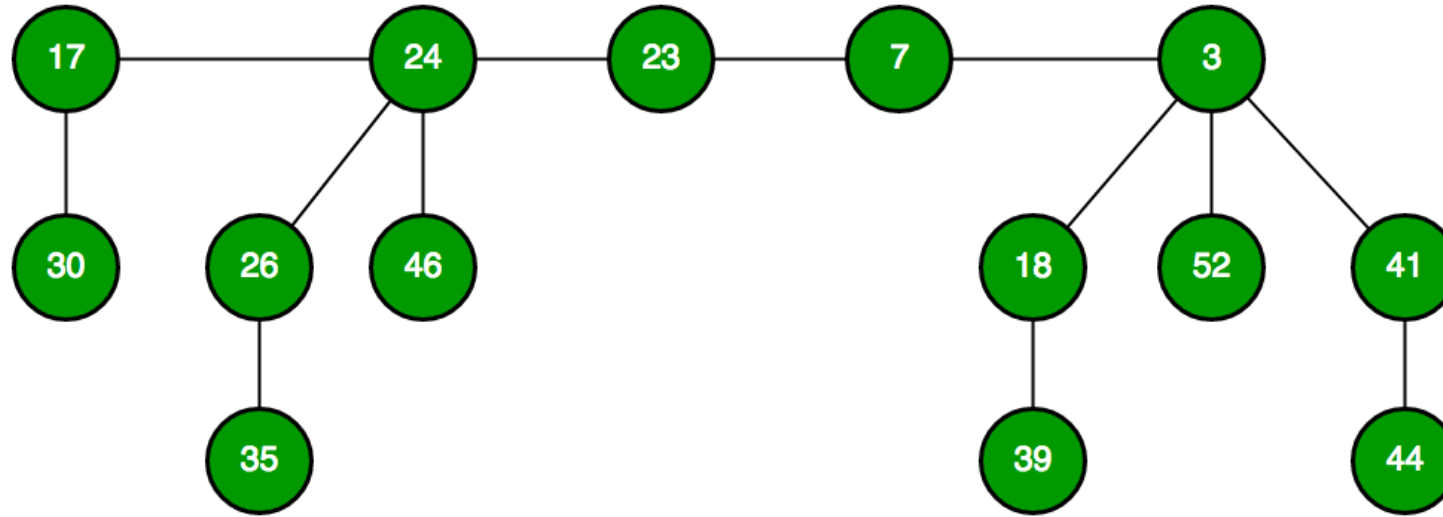
Sada sekvenčně vytvářených binomiálních stromů B_k

- Výška stromu B_k je k
- Strom B_k má 2^k uzlů
- V hloubce i je $\binom{k}{i}$ uzlů
- Extrém je hledán v cyklického horizontálním seznamu kořenových uzlů
- Po jeho odebrání je korigován příslušný B_k

Zdroje: https://en.wikipedia.org/wiki/Binomial_heap

<https://www.geeksforgeeks.org/memory-representation-of-binomial-heap/>

Fibonacciho halda



Zdroj: <https://www.geeksforgeeks.org/fibonacci-heap-set-1-introduction/>

- **Sada** sekvenčně vytvářených stromů s počtem uzlů, odpovídajícím (nejvýše) počtu členů Fibonacciho posloupnosti (1, 2, 3, 5, 8, 13, ...).
- Nový prvek je uložen do kořenové úrovně jako nová dílčí halda, která je dle potřeby zřetězena s ostatními.
- Extrém je hledán v cyklického horizontálním seznamu kořenových uzlů.
- Výhodou této struktury je nízká asymptotická složitost většiny běžných operací.

Zásobník, fronta a prioritní fronta v STL = adaptéry

STL: kontejnery, iterátory, algoritmy, funktory

Kontejnery: sekvenční, adaptéry (k sekvenčním), asociativní, neseřazené

Adaptéry: zásobník (Z), fronta (F), prioritní fronta (PF)

Z: `template <class T, class Container = deque<T> > class stack;`

F: `template <class T, class Container = deque<T> > class queue;`

PF: `template <class T, class Container = vector<T>, class Compare = less<Container::value_type> > class priority_queue;`

Prioritní fronta pomocí STL knihovny

[Výstup: 50, 30, 3]

```
#include <iostream>
#include <queue>
#include <vector>                                     // pro Min Heap

int main() {
    std::priority_queue<int> maxHeap; // Vytvoření prioritní fronty      (Max Heap)
    // std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap; (Min Heap)

    maxHeap.push(30);                                // Vložení prvků do prioritní fronty
    maxHeap.push(10);
    maxHeap.push(50);
    maxHeap.push(20);

    std::cout << "Nejvyšší prvek: " << maxHeap.top() << std::endl;      // Výpis MAX_1

    maxHeap.pop();                                    // Odstranění MAX_1

    std::cout << "Nový nejvyšší prvek: " << maxHeap.top() << std::endl;   // Výpis MAX_2

    std::cout << "Velikost prioritní fronty: " << maxHeap.size() << std::endl; // Velikost
    return 0; }
```

2x zásobník pomocí STL knihovny

[Výstup: 3, 2, 1]

```
#include <iostream>
#include <stack>

int main() {
    std::stack<int> myStack;

    // Vkládání do zásobníku
    myStack.push(1);
    myStack.push(2);
    myStack.push(3);

    // Výpis a odstranění obsahu zásobníku
    while (!myStack.empty()) {
        std::cout << myStack.top() << " ";
        myStack.pop();
    }

    return 0;
}
```

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> myStack;

    myStack.push_back(1);           // Vložení na konec fronty
    myStack.push_back(2);
    myStack.push_back(3);

    while (!myStack.empty()) {
        std::cout << myStack.back() << " "; // Výpis posledního
        myStack.pop_back();                // Odstranění posledního
    }

    return 0;
}
```

2x fronta pomocí STL knihovny

[Výstup: 1, 2, 3]

```
#include <iostream>
#include <queue>

int main() {
    std::queue<int> myQueue;

    // Vkládání do fronty
    myQueue.push(1);
    myQueue.push(2);
    myQueue.push(3);

    // Výpis a odstranění obsahu fronty
    while (!myQueue.empty()) {
        std::cout << myQueue.front() << " ";
        myQueue.pop();
    }
    return 0;
}
```

```
#include <iostream>
#include <queue>

int main() {
    std::deque<int> myQueue;

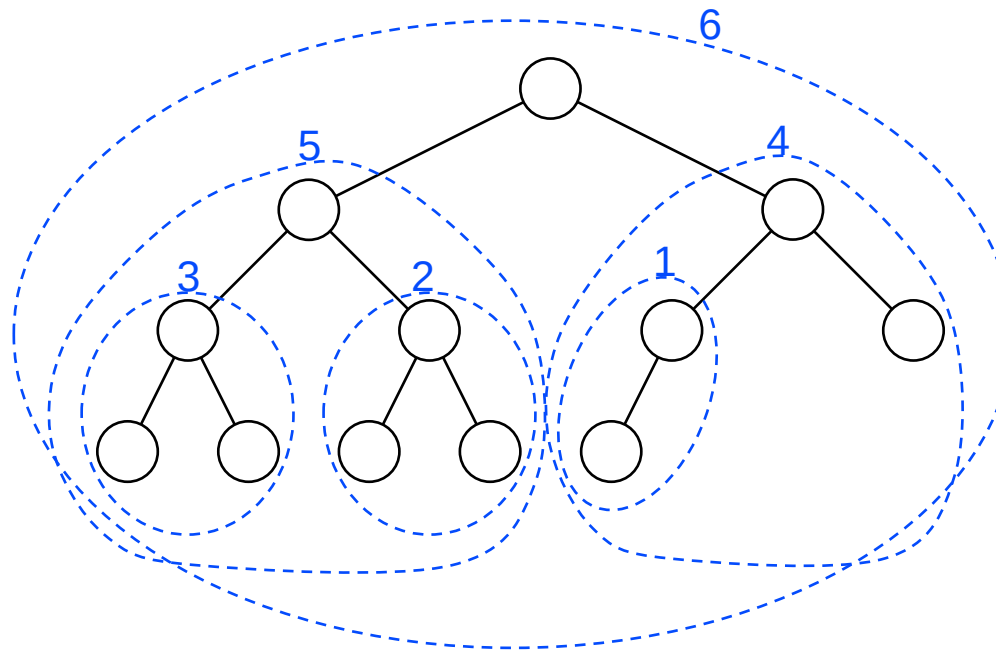
    myQueue.push_front(1);    // Vložení na začátek fronty
    myQueue.push_front(2);
    myQueue.push_front(3);

    while (!myQueue.empty()) {
        std::cout << myQueue.back() << " "; // Výpis posledního
        myQueue.pop_back();                // Odstranění posledního
    }

    return 0;
}
```

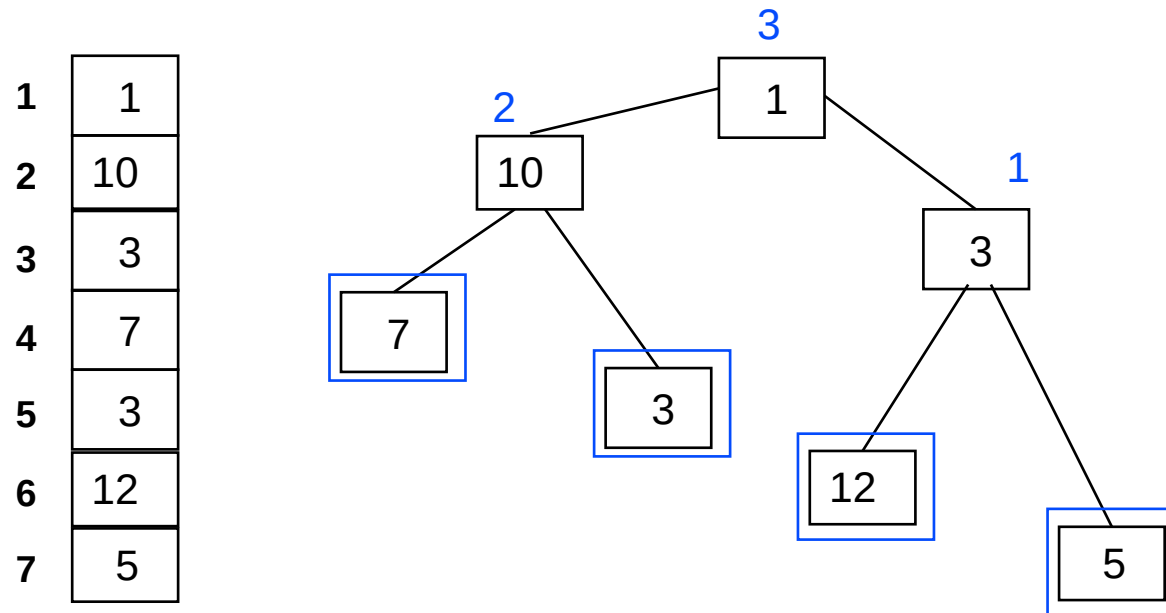

Vytváření haldy v poli zdola nahoru

- Haldu v poli lze efektivněji vytvořit přes dílčí haldy, které se zpětným průchodem polem a použitím algoritmu opravy haldy shora dolů (*fixDown*) postupně zvětšují
 - každou pozici v poli bereme jako kořen dílčí haldy
 - jestliže následníci $u[2k]$ a $u[2k+1]$ uzlu $u[k]$ jsou kořeny hald, pak haldu s kořenem $u[k]$ vytvoříme opravou shora dolů
 - opravy provádíme postupně pro k od $n/2$ do 1
- Příklad postupu:



Vytváření haldy v poli zdola nahoru

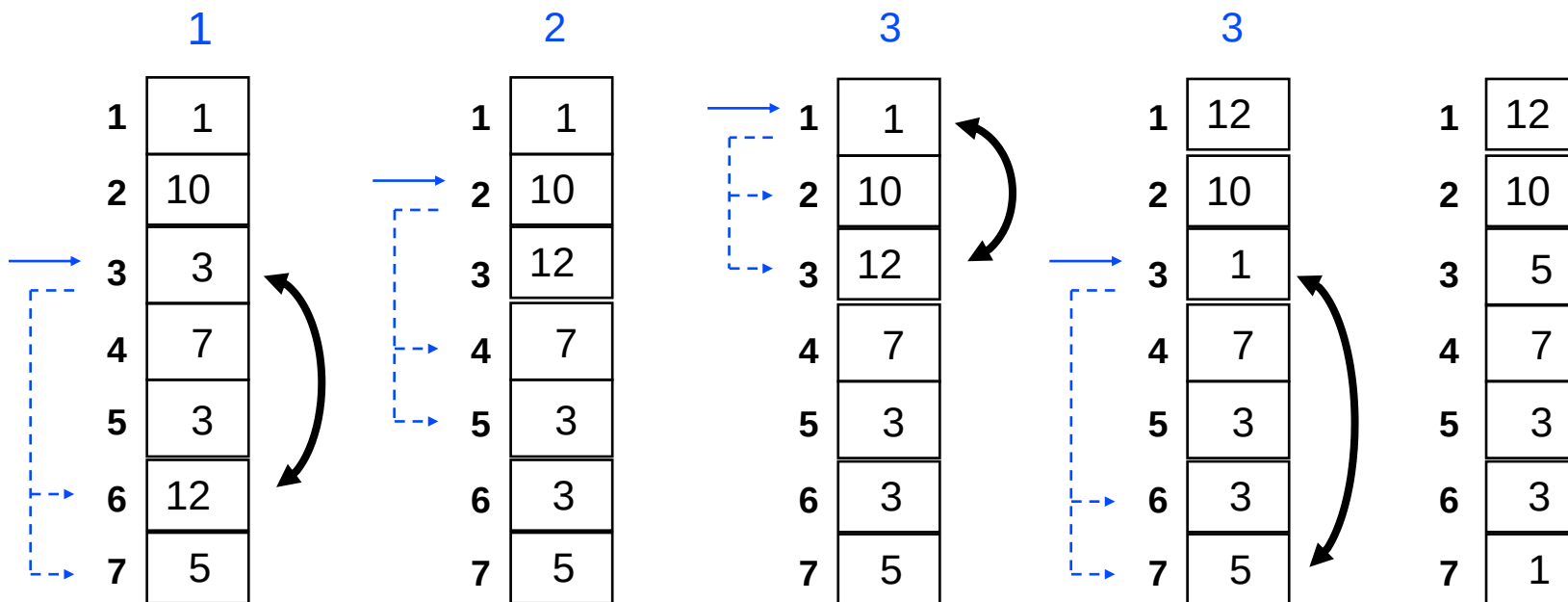
- Příklad:



- Každý z uzlů $u[4]$ až $u[7]$ je haldou (tvořenou jediným uzlem)
- Opravu haldy algoritmem **fixDown** postupně provádíme pro uzly $u[3]$, $u[2]$ a $u[1]$

Vytváření haldy v poli zdola nahoru

- Příklad



- Složitost vytváření haldy v poli zdola nahoru je $O(n)$, kde n je počet prvků pole (ověříme na cvičení)

Vytváření haldy v poli zdola nahoru

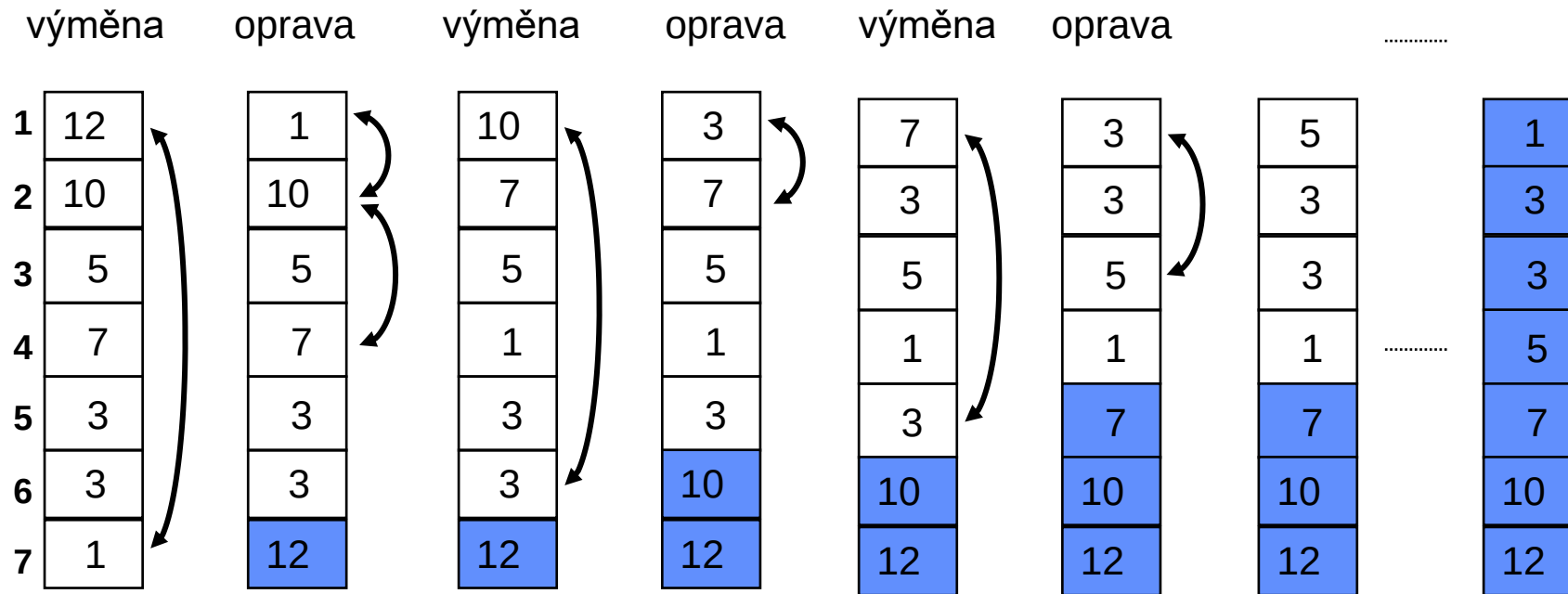
```
#include "pqueue.h"
#include <iostream>
using namespace std;
```

```
void createHeap(int *a, int n) {
    int k;
    for (k=n/2; k>=1; k--)
        fixDown(a, k, n);
}
```

```
int main() {
    int a[] = {0,1,10,3,7,3,12,5};
    int na = sizeof(a)/sizeof(int);
    createHeap(a, na-1);           //vytvoření haldy
    for (int i=0; i<na; i++)       //tisk haldy
        cout << a[i] << ' ';
    cout << endl;
    system("PAUSE");
    return 0;
}
```

Algoritmus řazení Heapsort

- Pole obsahující (max-) haldu lze upravit na (vzestupně) seřazené pole tak, že opakovaně vyměníme první prvek pole s posledním prvkem haldy, haldu opravíme shora dolů a pak zmenšíme.
- Příklad:



Řazení Heapsort

- Algoritmus Heapsort - shrnutí:
 - vytvoření max-haldy v poli
 - opakovaná výměna kořene haldy s posledním prvkem haldy,
 - oprava haldy shora dolů a zmenšení haldy
- Šablony:

```
template <class T>           //oprava haldy shora dolů
void fixDown(T *a, int k, int n) {
    int j;
    while (2*k <= n) {
        j = 2 * k;
        if (j<n && a[j]<a[j+1])
            j++;           //vyber většího ze synů
        if (a[k]>=a[j])
            break;
        swap(a[k], a[j]);
        k = j;
    }
}
```

Řazení Heapsort

- Šablony - pokračování:

```
template <class T>
void heapSort(T *a, int n) {
    int k;
    for (k=n/2; k>=1; k--)           //vytvoření haldy
        fixDown(a, k, n);
    while (n>1) {
        swap(a[1], a[n]);           //kořen haldy na konec
        fixDown(a, 1, --n);         //úprava haldy
    }
}
```

- Použití je v souboru p5\heapsort\main.cpp
- Program p5\heapsort\printheapsort.cpp průběžně vypisuje obsah řazeného pole

Řazení Heapsort

- Vlastnosti:

- časová složitost v nejlepším případě $\Omega(n \log n)$
- časová složitost v nejhorším případě $O(n \log n)$
- časová složitost $\Theta(n \log n)$
- paměťová složitost $O(1)$
- stabilita řazení **ne**

Algoritmy řazení, úvodní motivace

- Rozdělení
 - Vnitřní řazení (v paměti) a vnější řazení (na disku)

Jde o explicitní techniky používané v případech, kdy nelze vyhledávat pomocí *průběžně aktualizovaných* klíčů nebo indexů

- V případě SQL jazyka se může jednat o příkazy ORDER BY, GROUP BY, DISTINCT, UNION, DIFFERENCE apod.

Motivační příklad



Zdroj: <https://www.toptal.com/developers/sorting-algorithms>

Algoritmy řazení, shrnutí

- Řazení má smysl pouze v případě existence tělesa **úplně uspořádaných klíčů**
- Důležitá je časová složitost, významnou roli zde hraje i složitost paměťová
 - Na místě (**in-place**)
 - S využitím pomocné struktury (**out-of-place**)
- Stabilita (zachování relativního pořadí prvků se stejnými klíči)
 - **Stabilní**
 - **Nestabilní**

Základní „výkonové“ kategorie

$O(n^2)$ - sekvenční

- Bubble
 - Comb (proměnná vzdálenost, podobně jako u Shell sortu)
 - Cocktail/Shaker (oboustranný průchod daty, eliminace „želv“)
- Selection
- Insertion (prakticky aplikovatelný na data malého rozsahu)
 - Shell (proměnná vzdálenost, „deterministická randomizace“)

$O(n \log(n))$ – rozděl a panuj (stromové)

- Merge (vytvoření, seřazení a spojování dílčích seznamů)
- Quick (řazení zbytky dat vzhledem k pivotovi)
- Heap (halda – speciální binární strom, kde klíč rodiče > klíč potomků)

Počítací (specifické)

- Counting $O(n + k)$, k : počet indexů
- Radix $O(m * C(n))$, m : počet řádů/znaků, $C(n)$: vnitřní algoritmus
- Bucket (Bin) $O(m * C(n/m))$, m : počet přihrádek

Řazení

- Je dána množina $M = \{m_1, m_2, \dots, m_n\}$
- Prvek množiny se skládá ze dvou složek $m_j = [k_j, h_j]$
 - k_j je **klíč prvku**,
klíče k_j jsou z jistého univerza U , na němž je definována
relace úplného uspořádání $\leq$, resp. $\geq$
 - h_j je **hodnota prvku**.
- **Vzestupné řazení:** vytvoření takové n -tice (**permutace**) prvků množiny M ,
aby její prvky tvořily monotonně rostoucí posloupnost $k_1 \leq k_2 \leq k_3 \dots k_{n-1} \leq k_n$,
- **Sestupné řazení:** vytvoření takové n -tice aby její prvky tvořily monotonně
klesající posloupnost () tj. $k_1 \geq k_2 \geq k_3 \dots k_{n-1} \geq k_n$,

Algoritmy řazení

- Reprezentace prvku množiny M

```
• struct Prvek{  
    TK klic;  
    TH hodnota;  
};
```

- Reprezentace množiny M
 - **Spojový seznam**
 - **Pole**

Algoritmy řazení

- Pokud algoritmus řazení zapíšeme jako šablonu funkce, je použitelný pro jakýkoliv typ prvků, pro který je definován operátor použitý v šabloně pro porovnání klíčů

```
template <class T>  
void sort(T *a, int n) { ... }  
(stejnou hlavičku budou mít všechny další algoritmy řazení)
```

- Ve většině algoritmů řazení budeme prvky porovnávat operátorem <
- Šablony funkcí tedy budou použitelné pro libovolné pole, pro jehož prvky (přesněji řečeno pro typ jeho prvků) je definován operátor <

Časová složitost algoritmů řazení

- Při určování časové složitosti řazení vycházíme z počtů testů a počtu přesunů v závislosti na počtu řazených prvků

Složitost budeme určovat pro:

nejlepší případ: určuje složitost Ω , např. $\Omega(n)$

nejhorší případ: určuje složitost O , např. $O(n^2)$

průměrný případ

Pokud jsou složitosti v nejlepším a nejhorším případě stejné (např. n^2), můžeme složitost algoritmu vyjádřit notací Θ (viz 2. přednáška)

Pokud složitost Ω a O nejsou stejné, vyjadřujeme složitost algoritmu notací O (nejhorší případ)

Paměťová složitost algoritmů řazení

- Pro porovnávání algoritmů řazení je důležité také to, kolik paměti „**navíc**“ algoritmus potřebuje pro seřazení n prvků.
- Tuto vlastnost vyjadřuje *paměťová složitost*.
- Závislost velikosti paměti „navíc“ vyjadřujeme podobnými notacemi, jako časovou složitost, omezíme se ale jen na nejhorší případ.
- Algoritmy řazení, které dále uvedeme, mají jednu z následujících paměťových složitostí:
 - **konstantní**, tj $O(1)$, což znamená, že algoritmus pro seřazení libovolného počtu prvků potřebuje navíc jen pevně daný počet proměnných
 - **lineární**, tj. $O(n)$, což znamená, že algoritmus pro seřazení pole délky n potřebuje další paměť, jejíž velikost se úměrná počtu prvků n
 - **logaritmická**, tj $O(\log n)$, což znamená, že algoritmus pro seřazení pole délky n potřebuje další paměť, jejíž velikost se úměrná logaritmu počtu prvků n

Stabilita řazení

- Stabilní řazení: zachovává relativní pořadí prvků se stejnými klíči

stabilní řazení

Novak	Adam
Vesely	Josef
Rada	Josef
Novak	Josef
Kratky	Petr
Vesely	Petr

Kratky	Petr
Novak	Adam
Novak	Josef
Rada	Josef
Vesely	Josef
Vesely	Petr

Kratky	Petr
Novak	Josef
Novak	Adam
Rada	Josef
Vesely	Petr
Vesely	Josef

nestabilní řazení

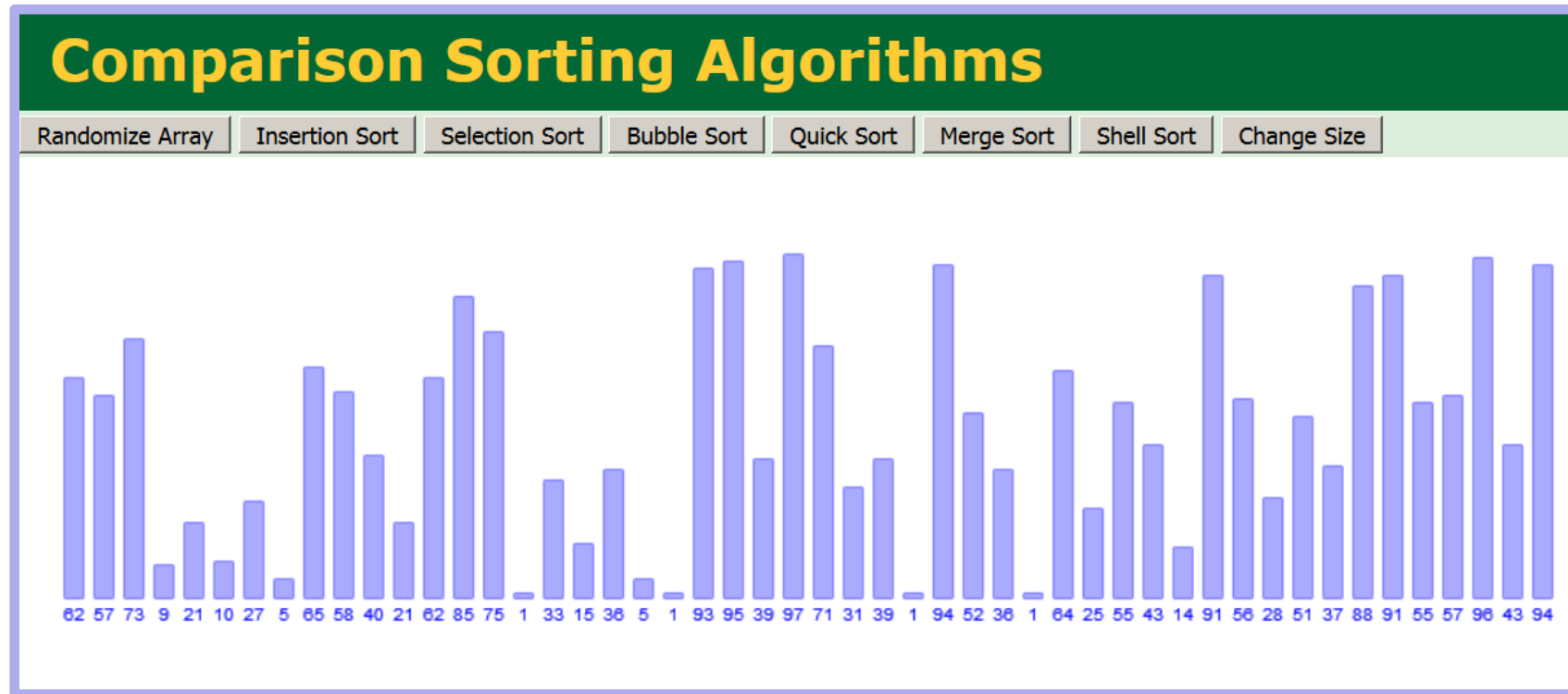
- Stabilní řazení je důležitá v případech, kdy prvky řadíme opakovaně podle různých klíčů (v předchozím příkladě nejprve podle jména a pak podle příjmení)

Algoritmy řazení

- Je známo mnoho algoritmů řazení, jejichž asymptotická časová složitost se pohybuje od $O(n^2)$ přes $O(n \log n)$ až do $O(n)$
- Pozor ale:
 - asymptoticky rychlejší může mít horší paměťovou složitost
 - asymptoticky pomalejší algoritmus může malý počet prvků seřadit rychleji než asymptoticky rychlejší algoritmus
- Probereme (zopakujeme) zde tyto algoritmy řazení:

– Selectsort	}	$O(n^2)$
– Insertsort		
– Bubblesort		
– Mergesort	}	$O(n \log n)$
– Heapsort		
– Quicksort		
– Radixsort		$O(n.m)$

Řazení, vizualizace 1 (principy)



Zdroj: <https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

Řazení, vizualizace 2 (pseudokód a testy)



Zdroj: <https://visualgo.net/sorting>

Řazení Selectsort

- Princip: řazení výběrem, opakované hledání nejmenšího prvku ve zmenšujícím se úseku pole a jeho zařazení na správné místo.
- Šablona:

```
template <class T>
void selectSort(T a[], int n) {
    int i, j, imin;
    for (i=0; i<n-1; i++) {
        imin = i;
        for (j=i+1; j<n; j++) /* výběr nejmenšího */
            if (a[j] < a[imin]) imin = j;
        if (imin!=i)
            swap(a[imin], a[i]); /* nejmenší na začátek */
    }
}
```

Řazení Selectsort

- Vlastnosti:
 - časová složitost v nejlepším případě $\Omega(n^2)$
 - časová složitost v nejhorším případě $O(n^2)$
 - časová složitost $\Theta(n^2)$
 - paměťová složitost $O(1)$
 - stabilita řazení ano
- Co může způsobit nestabilitu: když operaci $<$ nahradíme operací $\leq$

Řazení Insertsort

- Princip: opakované vkládání do již seřazené části pole vkládáme nový prvek
- Šablona:

```
template <class T>
void insert(T a[], int k, T x) {
    int i;
    for(i=k-1; i>=0 && x<a[i];i--)a[i+1]=a[i]; // posun
    a[i+1] = x;      // vložení
}
template <class T>
void insertSort(T *a, int n) {
    int k;
    for (k=1; k<n ; k++)
        insert(a, k, a[k]);
}
```

Řazení Insertsort

- Vlastnosti:

- časová složitost v nejlepším případě $\Omega(n)$ pro seřazené pole
- časová složitost v nejhorším případě $O(n^2)$
- časová složitost $O(n^2)$
- paměťová složitost $O(1)$
- stabilita řazení ano

- Co může způsobit nestabilitu: když operaci $<$ nahradíme operací $\leq$

Řazení Bubblesort

- Princip: opakované zaměňování porovnáme sousedy a když nesplňují požadovanou relaci, vyměníme je
- Šablona:

```
template <class T>
void bubbleSort(T a[], int n) {
    int i;
    bool vymeneno;
    do {
        vymeneno = false;
        for (i=1; i<n; i++)
            if (a[i] < a[i-1]) {
                swap(a[i], a[i-1]);
                vymeneno = true;
            }
        n--;
    } while (vymeneno);
}
```

Řazení Bubblesort

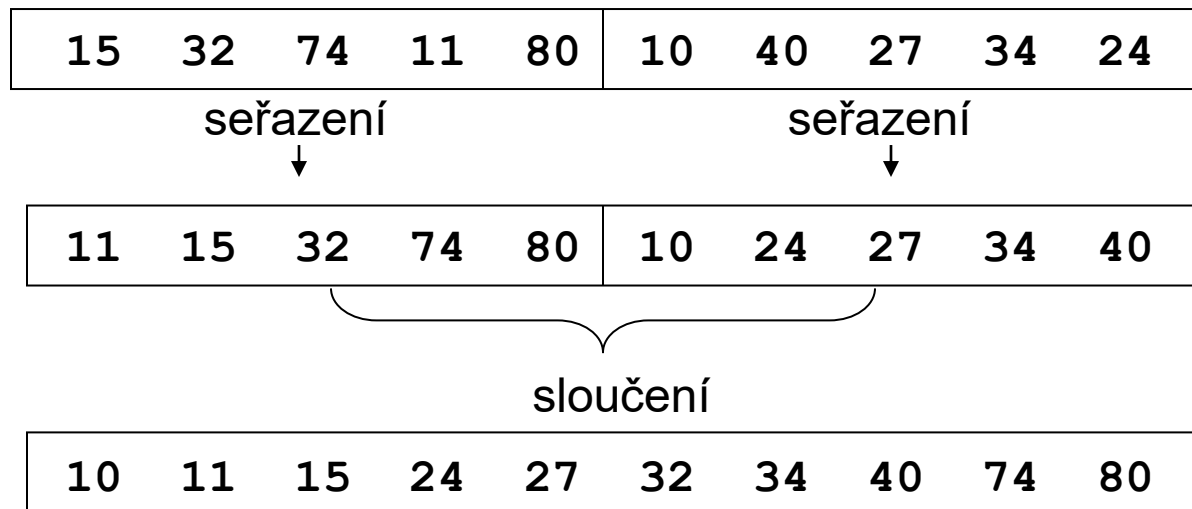
- Vlastnosti:

- časová složitost v nejlepším případě $\Omega(n)$ pro seřazené pole
- časová složitost v nejhorším případě $O(n^2)$
- časová složitost $O(n^2)$
- paměťová složitost $O(1)$
- stabilita řazení ano

- Co může způsobit nestabilitu: když operaci $<$ nahradíme operací $\leq$

Řazení Mergesort (rekurzivní)

- Princip: opakované slučování
 - řazený úsek pole rozděl na dvě části
 - seřaď levý úsek a pravý úsek
 - přepiš řazený úsek pole sloučením levého a pravého úseku



Řazení Mergesort (rekurzivní)

- Šablony:

```
template <class T>          // slučování
void merge(T a[], T b[],
           int levy, int pravy, int poslPravy) {
    int poslLevy = pravy-1;
    int i = levy;
    int prvniLevy = levy;

    while (levy<=poslLevy && pravy<=poslPravy)
        b[i++] = (a[levy] < a[pravy])? a[levy++]
        :a[pravy++]; // menší do b

    // kopírování zbytku
    while (levy<=poslLevy) b[i++] = a[levy++];
    while (pravy<=poslPravy) b[i++] = a[pravy++];
}
```

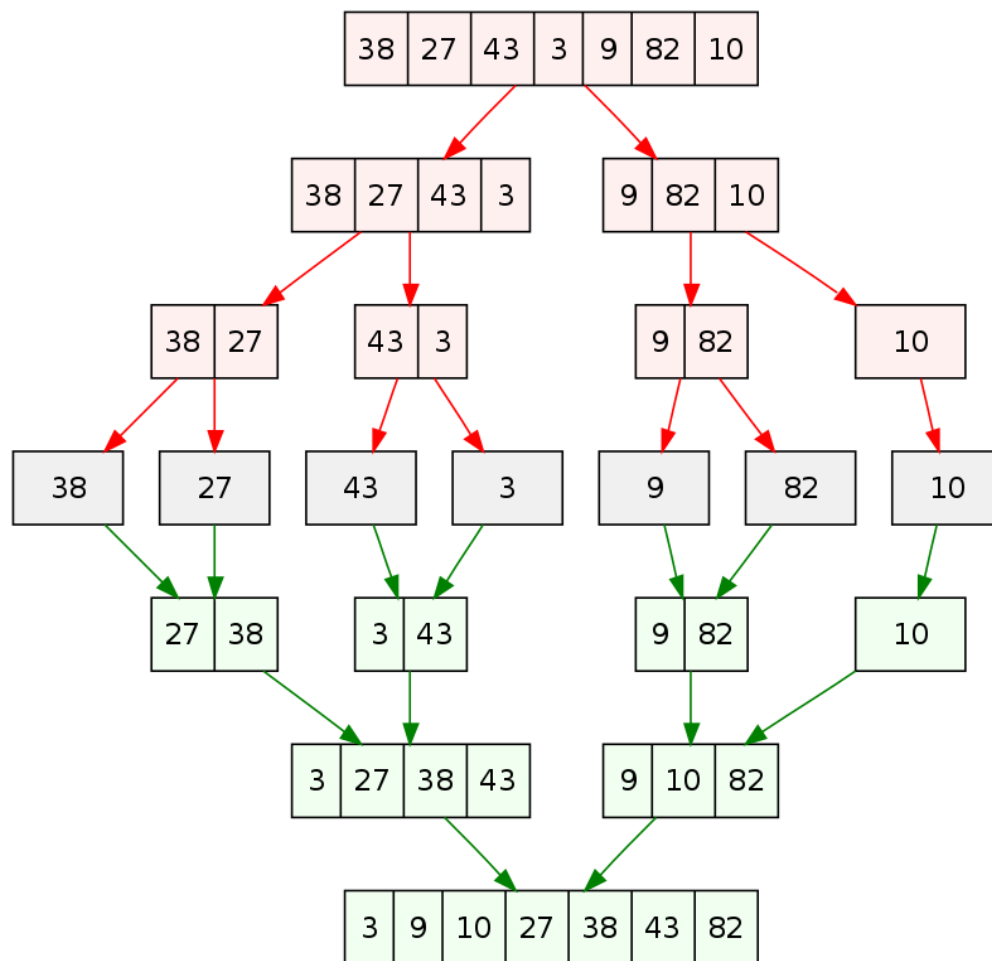
Řazení Mergesort (rekurzivní)

- Šablony (pokračování):

```
template <class T>
void mergeSortRek(T a[], T pom[], int prvni, int posl) {
    int i;
    if (prvni < posl) {
        int stred = (prvni + posl) / 2;
        mergeSortRek(a, pom, prvni, stred); // seřad' vlevo
        mergeSortRek(a, pom, stred + 1, posl); // seřad' vpravo
        merge(a, pom, prvni, stred + 1, posl); // sloučení
        // kopírování z pole pom do pole a
        for (i = prvni; i <= posl; i++) a[i] = pom[i];
    }
}

template <class T>
void mergeSort(T a[], int n) {
    T *pom = new T[n]; // pomocné pole
    mergeSortRek(a, pom, 0, n - 1);
    delete(pom);
}
```

Rekurzivní Merge sort, příklad



Zdroj: <https://hopegiometti.medium.com/basic-solutions-to-sorting-algorithms-merge-sort-fde49bf6ad41>

Řazení Mergesort (rekurzivní)

- Vlastnosti:

- časová složitost v nejlepším případě $\Omega(n \log n)$
- časová složitost v nejhorším případě $O(n \log n)$
- časová složitost $\Theta(n \log n)$
- paměťová složitost $O(n)$
- stabilita řazení ano

- Co může způsobit nestabilitu: když operaci $<$ ve funkci *merge* nahradíme operací $\leq$

Nerekurzivní Mergesort

- **Rekurzivní** Mergesort se volá rekurzivně tak dlouho, dokud délka úseku pole není 1; teprve pak začne slučování sousedních úseků do dvakrát většího úseku v pomocném poli, který je třeba zkopírovat do původního pole
- Rozdělení pole na úseky, které se postupně slučují, je dáno postupným půlením úseků pole **shora dolů**.
- **Nerekurzivní** (iterační) algoritmus Mergesort postupuje **zdola nahoru**:
 - pole *a* se rozdělí na dvojice úseků délky 1, které se sloučí do seřazených úseků délky 2 v pomocném poli *pom*
 - dvojice sousedních seřazených úseků délky 2 v poli *pom* se sloučí do seřazených úseků délky 4 v poli *a*
 - dvojice sousedních úseků délky 4 v poli *a* se sloučí do seřazených úseků délky 8 v poli *pom*
 - atd.
 - tento postup se opakuje, pokud délka úseku je menší než velikost pole
 - skončí-li slučování tak, že výsledek je v pomocném poli *pom*, je třeba jej zkopírovat do původního pole *a*

Nerekurzivní Mergesort -příklad

a

49 62 | 21 70 | 89 99 | 21 76 | 53 40 | 87 70 | 32 70 | 24 93 | 90 65 | 90

pom

49 62 21 70 | 89 99 21 76 | 40 53 70 87 | 32 70 24 93 | 65 90 90

a

21 49 62 70 | 21 76 89 99 | 40 53 70 87 | 24 32 70 93 | 65 90 90

pom

21 21 49 62 70 76 89 99 | 24 32 40 53 70 70 87 93 | 65 90 90

a

21 21 24 32 40 49 53 62 70 70 70 76 87 89 93 99 | 65 90 90

pom

21 21 24 32 40 49 53 62 65 70 70 70 76 87 89 90 90 93 99

Řazení Mergesort (nerekurzivní)

- Šablony:

```
template <class T>
void merge(T a[], T b[],          //slučování
           int levy, int pravy, int poslPravy) {
    int poslLevy = pravy-1;
    int i = levy;
    int prvniLevy = levy;
    // výběr menšího vlevo a vpravo
    while (levy<=poslLevy && pravy<=poslPravy)
        b[i++] = (a[levy]<a[pravy])? a[levy++] :a [pravy++];
    // kopírování zbytků
    while (levy<=poslLevy) b[i++] = a[levy++];
    while (pravy<=poslPravy) b[i++] = a[pravy++];
}

inline int min(int x, int y) {
    if (x<y) return x;
    else    return y;
}
```

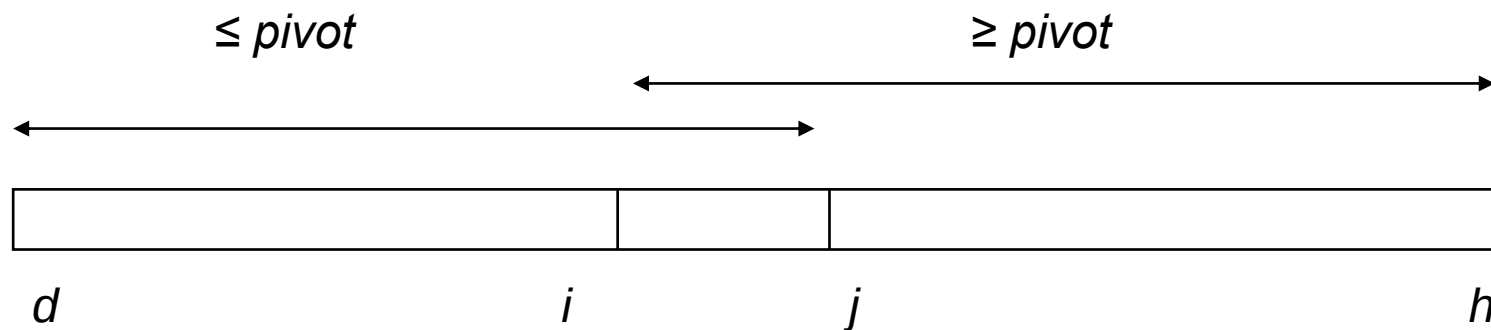
Řazení Mergesort (nerekurzivní)

- Šablony – pokračování

```
template <class T>
void mergeSort(T a[], int n) {
    T *pom = new T[n],           //pomocné pole
    *odkud = a, *kam = pom;
    int delkaUseku = 1;
    int posl = n-1, i;
    while (delkaUseku<n) {       //dělení na úseky
        int levy = 0;
        while (levy<=posl) {
            int pravy = levy+delkaUseku;
            merge(odkud, kam,
                  levy, min(pravy, n),
                  min(pravy+delkaUseku-1,posl));
            levy = levy+2*delkaUseku;    //další úseky
        }
        delkaUseku = 2*delkaUseku;
        int *p = odkud; odkud = kam; kam = p; //záměna plí
    }
    if (odkud!=a)
        for (i=0; i<n; i++) a[i] = pom[i]; //kopírování do a
}
```

Řazení Quicksort

- Princip:
 - pro řazený úsek pole se vybere vhodná hodnota klíče dále zvaná **pivot**
 - prvky v řazeném úseku se přeskupí tak, aby tvořily tři části:
 - prvky s klíčem menším než pivot
 - prvky s klíčem rovným *pivotu*
 - prvky s klíčem větším než *pivot*
 - pak se tímtéž algoritmem (rekurzivně) seřadí levý a pravý úsek



- Přeskupení řazeného úseku pole provede procedura **partition**

Řazení Quicksort

- Šablony:

```
template <class T>
void partition(T a[],
              //dolní mez úseku int d,
              int& i, int& j,
              //horní mez úseku int h)
{
    i = h; j = d;
    T pivot = a[(d+h)/2];    // výběr pivota
    while (i>j) {
        while (a[i]>pivot) i--; // výběr kandidátů na výměnu
        while (a[j]<pivot) j++;
        swap(a[i], a[j]);    // výměna prvků
        if (i>d) i--;
        if (j<h) j++;
    }
}
```

Řazení Quicksort

- Šablony:

```
template <class T>
void qsort(T a[], int d, int h) {
    if (d < h) {
        int i, j;
        partition(a, d, i, j, h);
        qsort(a, d, i);           //třídění dolní části
        qsort(a, j, h);          //třídění horní části
    }
}

// zahájení třídění
template <class T>
void quickSort(T a[], int n) {
    qsort(a, 0, n-1);
}
```

Řazení Quicksort (nerekurzivní)

- Algoritmus Quicksort lze nerekurzivně zapsat s použitím zásobníku
- Po každém rozdělení procedurou *partition* se algoritmus věnuje řazení jednoho úseku a meze druhého úseku uloží do zásobníku

Řazení Quicksort (nerekurzivní)

- Šablony:

```
template <class T>
void quickSort(T a[], int n) {
    int d = 0, h = n-1, i, j;
    Stack<int> stc;
    stc.push(d);
    stc.push(h);
    do {
        h = stc.pop();
        d = stc.pop();
        do {
            partition(d, i, j, h);
            if (i-d < h-j) { //delší úsek do zásobníku
                stc.push(j);
                stc.push(h);
                h = i;
            } else {
                stc.push(d);
                stc.push(i);
                d = j;
            }
        } while (d<h);
    } while (!stc.empty());
}
```


Řazení Quicksort

- Vlastnosti:

- časová složitost v nejlepším případě $\Omega(n \log n)$
- časová složitost v nejhorším případě $O(n^2)$
- časová složitost v průměrném případě $O(n \log n)$
- paměťová složitost $O(\log n)$
- stabilita řazení ne

Řazení Casesort

- Princip:
 - klíč je posloupnost m číslic,
 - v prvním průchodu vložíme prvky pole podle poslední číslice do m přihrádek (**front**),
 - pak z přihrádek prvky vybereme a vytvoříme z nich upravené pole, jehož prvky budou seřazeny podle poslední číslice,
 - uvedený postup opakujeme pro druhou a další číslici zprava.
- Tento algoritmus řazení používaly v dávných dobách děroštitkové stroje
- Alternativní názvy algoritmu: **Radixsort** (radix = řádový, základ), **přihrádkové řazení**

Řazení Casesort (Radix-Sort)

- Postup řazení metodou Radix-Sort při základu 10

Řazené pole : 125, 082, 625, 324, 715, 304, 065, 706, 102,

Seřazené pole : 065, 082, 102, 125, 304, 324, 625, 706, 715

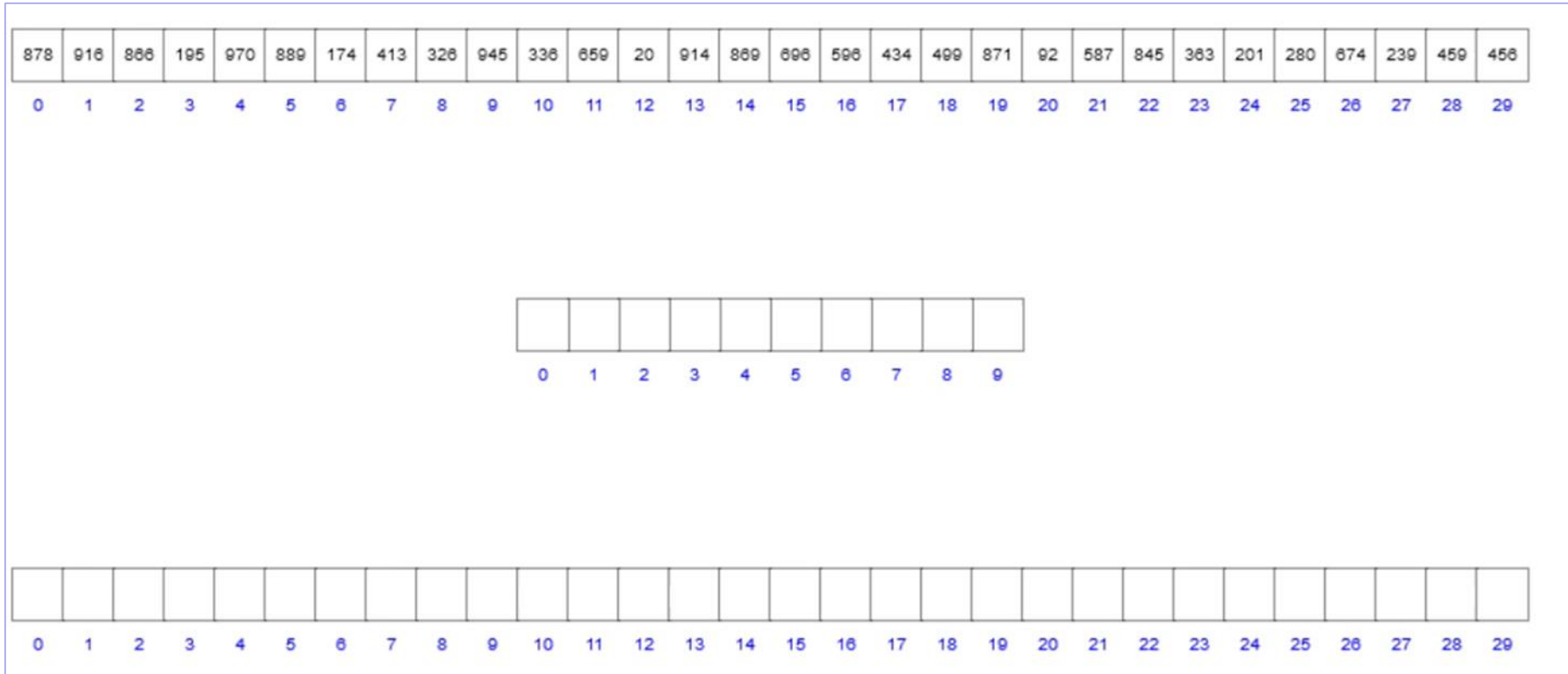
	třídy									
řád	0	1	2	3	4	5	6	7	8	9
0			082		324	125	706			
			102		304	625				
						715				
						065				
1	102	715	324				065		082	
	304		125							
	706		625							
2	065	102		304			625	706		
	082	125		324				715		

Řazení Casesort

- Funkce caseSort:

```
//výběr r-té šestnáctkové cifry z x
static int hexCislice(int x, int r) {
    return (x >> 4*r) & 0xF;    }
//add, remove a empty jsou operace dat. typu Queue
void caseSort(int a[], int n) {
    Queue<int> prihradky[16]; // pole 16-ti front
    int r, j, c;              //roztrídění do přihrádek
    for (r=0; r<8; r++) {
        for (j=0; j<n; j++)
            prihradky[hexCislice(a[j],r)].add(a[j]);
        j = 0; //spojení přihrádek - kopírování do a
        for (c=0; c<16; c++)
            while (!prihradky[c].empty()) {
                a[j++] = prihradky[c].front();
                prihradky[c].remove();
            }
    }
}
```

Radix sort



Zdroj: <https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

Řazení Casesort

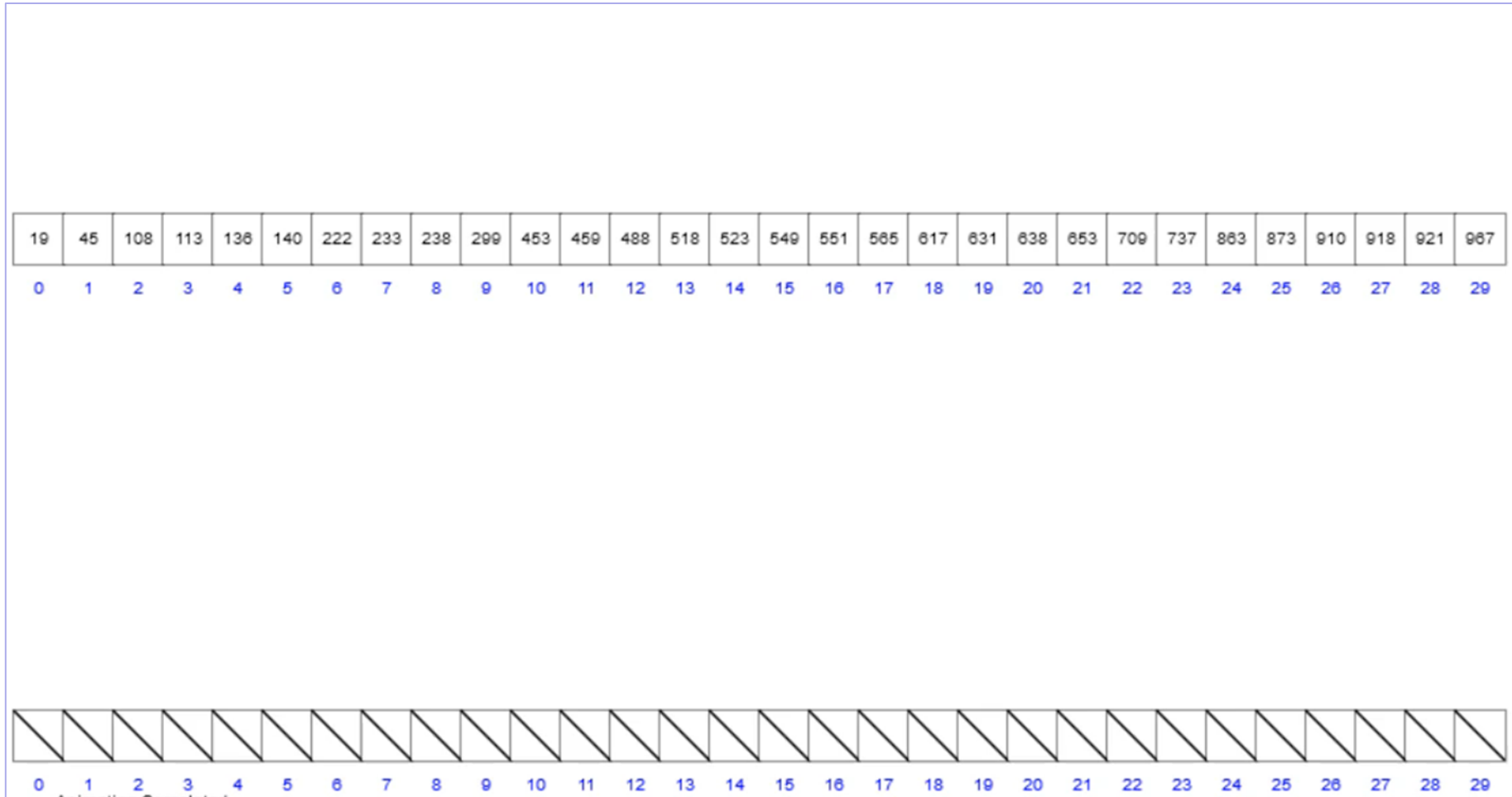
- Vlastnosti:

- časová složitost v nejlepším případě $\Omega(m.n)$
- časová složitost v nejhorším případě $O(m.n)$
- časová složitost $\Theta(m.n)$
- paměťová složitost $O(n)$
- stabilita řazení ano
- kde m je počet číslic klíče

- Poznámky:

- pro velký počet číslic m může m být i větší než $\log n$
 - čím je menší počet bitů pro číslici, tím je počet číslic větší
- Řazení Casesort je adresním řazením, při kterém se neporovnávají klíče, ale klíč je přímo využit pro umístění prvku v seřazeném poli
 - Všechny ostatní algoritmy řazení představují asociativní řazení, při kterém se klíče řazených prvků porovnávají

Bucket sort



Zdroj: <https://www.cs.usfca.edu/~galles/visualization/BucketSort.html>

Složitost řadících algoritmů

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$

Zdroj: <http://bigocheatsheet.com/>